TEC

Estructuras de Datos(IC-2001) y Algoritmos(CA-3125)

Proyecto #1A (obligatorio)

Fase Orbital: motor de juego multijugador en C+Allegro con estadísticas, construido sobre estructuras lineales enlazadas

Dato	Valor
Profesor	Víctor Garro Abarca
Institución / código del curso	TEC CA-3125 IC-2001
Periodo lectivo / grupo	2do semestre 2026
Modalidad	TRIOS
Valor en la nota final	15%
Lenguaje y biblioteca gráfica	Visual C++2026 con Allegro 5 (versión 5.2.x)
Fecha de publicación	24/9/2026
Fecha de entrega del proyecto y documentación Técnica y de Usuario	29/10/2026 versión completa en un Drive con permisos. 5 semanas.
Entrega del video de defensa(Obligatorio)	29/10/2026. 5 semanas.

Contenido

1. Descripción general 4
1.1 Relación con el temario 4
2. Objetivos de aprendizaje 4
3. Reglas del juego 5
3.1 Espacio y tiempo de simulación 5
3.2 Fases de color 5
3.3 Enemigos 6
3.4 Puntaje, combo y pulso 6
3.5 Daño al núcleo 6
3.6 Oleadas y generación pseudoaleatoria 6
3.7 Orden de procesamiento de cada tick 7
4. Estructuras de datos obligatorias 7
4.1 Requisitos específicos 8
5. Modo multijugador 8
5.1 Modo por turnos (obligatorio) 8
5.2 Jugador automático (bot, obligatorio) 9
5.3 Modo cooperativo simultáneo (opcional, bonificación) 9
6. Módulo de estadísticas 9
6.1 Métricas por jugador y partida 9
6.2 Bitácora de eventos 10
6.3 Estadísticas históricas 10
6.4 Ranking 10
6.5 Persistencia 10
7. Interfaz y ejecución 11
7.1 Modos de ejecución 11
7.2 Línea de comandos 11
7.3 Formato del script de acciones 11
7.4 Requisitos de la interfaz Allegro 5 11
8. Arquitectura sugerida 12
9. Restricciones técnicas 13
10. Casos de prueba mínimos 13
11. Entregables 14
12. Rúbrica de revisión 15

12.1 Resumen de pesos 15
12.2 Documentación (25 %) 15
12.3 Ejecución (75 %) 16
12.4 Niveles de desempeño 17
12.5 Bonificación y penalizaciones 17
13. Cronograma sugerido 18
14. Normas académicas 18
Anexo A. Interfaces de referencia 18
Anexo B. Glosario 19
Anexo C. Videos Demostrativos/Explicativos 20

1. Descripción general

El proyecto consiste en implementar el motor de **Fase Orbital**, un juego arcade de disparos en el que una nave orbita un núcleo y lo defiende de enemigos de tres colores. El juego se presenta con una interfaz gráfica en **Allegro 5**, pero el énfasis del curso está en las estructuras de datos que lo sostienen: todas las colecciones dinámicas del motor deben implementarse a mano con nodos y punteros.

Sobre ese motor se construyen dos capacidades: un **modo multijugador por turnos** para 1 a 6 jugadores (humanos o bots), gestionado con una lista circular doblemente enlazada, y un **módulo de estadísticas** que registra el desempeño de cada jugador, lo persiste en archivo y calcula indicadores (media, mediana, desviación estándar, precisión, ranking) usando algoritmos sobre listas enlazadas.

El juego se basa en el prototipo jugable Fase Orbital publicado por el profesor. Esta especificación normaliza sus reglas en unidades de simulación y en ticks para que la ejecución sea determinista y evaluable.

1.1 Relación con el temario

Tema del curso	Dónde se aplica en el proyecto
Listas simplemente enlazadas	Lista de balas, bitácora de eventos, historial de partidas
Listas doblemente enlazadas	Lista de enemigos (eliminación e inserción arbitraria en O(1))
Listas circulares	Ronda de jugadores en el modo multijugador
Manejo de punteros simples y dobles	Eliminación durante recorrido con puntero a puntero (Nodo**)
Colas	Cola de aparición de enemigos por oleada
Copia profunda y gestión de memoria	Réplica de la oleada para cada jugador; cero fugas de memoria
Ordenamiento y análisis de complejidad	Merge sort sobre lista enlazada, inserción ordenada, mediana

2. Objetivos de aprendizaje

#	Objetivo	Evidencia esperada
O1	Implementar listas simples, dobles y circulares con nodos y punteros sin contenedores de la biblioteca estándar.	Clases propias con pruebas que cubren casos borde (vacía, un nodo, cabeza, cola).
O2	Eliminar e insertar nodos durante un recorrido sin invalidar punteros.	Balas y enemigos eliminados en el mismo ciclo que se recorren, sin saltos ni accesos inválidos.
O3	Implementar una cola enlazada y su copia profunda.	Constructor de copia y operador de asignación de ColaSpawn correctos.

#	Objetivo	Evidencia esperada
O4	Modelar turnos con una lista circular doblemente enlazada.	Rotación en ambos sentidos y eliminación del jugador actual.
O5	Aplicar algoritmos de ordenamiento y estadística sobre listas.	Merge sort O(n log n), mediana, desviación estándar y ranking.
O6	Gestionar memoria dinámica de forma segura.	Reporte de Valgrind o AddressSanitizer sin fugas ni errores.
O7	Analizar la complejidad temporal de cada operación.	Tabla de complejidades en el documento técnico.

3. Reglas del juego

3.1 Espacio y tiempo de simulación

El mundo es un plano con el núcleo en el origen (0, 0). Todas las distancias se expresan en unidades (u) y el tiempo en **ticks**; un tick equivale a 1/30 de segundo en el modo interactivo. Los ángulos se miden en grados, en sentido antihorario, en el rango [0, 360).

Parámetro	Valor
Radio del núcleo	20 u
Radio del anillo (órbita de la nave)	100 u
Radio de aparición de enemigos	300 u
Radio de eliminación de balas	320 u
Vida inicial del núcleo	100 (máximo 100)
Reparación al completar una oleada	+8
Giro máximo de la nave	6° por tick, por el camino angular más corto
Ángulo inicial de la nave en cada turno	270°
Disparo automático	Una bala cada 4 ticks (cuando tick mod 4 = 0)
Velocidad de bala	9 u/tick, en dirección radial hacia afuera
Velocidad base de enemigos en la oleada n	v = 0,8 + 0,08·n u/tick

3.2 Fases de color

Existen tres colores: **C** (cian), **M** (magenta) y **A** (ámbar). La nave tiene en todo momento una fase activa y cada bala hereda la fase vigente al momento de dispararse. Cuando una bala impacta a un enemigo:

- **Color correcto:** el enemigo pierde 1 punto de vida; si llega a 0 muere, suma puntos y el combo aumenta en 1.
- **Color incorrecto:** el enemigo retrocede 9 u (r = r + 9) y el combo del jugador vuelve a 0.

- En ambos casos la bala desaparece. Una bala impacta como máximo a un enemigo: el primero de la lista con el que colisione.

3.3 Enemigos

Cada enemigo se representa en coordenadas polares (r, θ). Aparece en r = 300 y avanza hacia el núcleo.

Tipo	Desde oleada	Vida	Radio	Velocidad radial	Comportamiento	Puntos
Dron	1	1	6	1,00·v	Avance radial recto	100
Girador	2	1	6	0,85·v	Espiral: ω = ±(0,9 + 0,05·n) °/tick	150
Divisor	3	1	8	0,80·v	Al morir inserta 2 Mini inmediatamente después de él en la lista	200
Mini	Por división	1	4	1,25·v	Espiral: ω = −1,7 y +1,7 °/tick; θ del padre ∓ 4°; colores = los dos distintos al del padre	75
Tanque	4	3	10	0,50·v	Avance radial recto	400
Camaleón	5	1	7	0,85·v	Cambia de color cada 42 ticks en el ciclo C → M → A → C	300

3.4 Puntaje, combo y pulso

- **Multiplicador:** mult = mín(8, 1 + ⌊combo / 6⌋). Puntos por baja = puntos del tipo × mult.
- **Carga de pulso:** +5 por baja (+14 si es Tanque), con máximo 100. Solo se puede activar con carga = 100; al activarlo vuelve a 0.
- **Onda de pulso:** nace en r = 20 y crece 12 u/tick. Destruye, sin importar el color, a todo enemigo con |r − r_onda| < radio + 3. Las bajas por pulso suman puntos pero no aumentan el combo ni la carga. La onda desaparece al superar r = 320.

3.5 Daño al núcleo

Evento	Condición	Efecto
Enemigo alcanza el núcleo	r ≤ 20 + 0,4·radio	Núcleo −10 (−22 si es Tanque); enemigo eliminado; combo = 0
Enemigo choca con la nave	Distancia euclídea a la nave < radio + 5	Núcleo −12; enemigo eliminado; combo = 0
Núcleo agotado	Vida ≤ 0	El jugador queda eliminado de la partida

3.6 Oleadas y generación pseudoaleatoria

Para que todos los grupos obtengan exactamente las mismas oleadas con la misma semilla, se usa el generador congruencial lineal siguiente (no se permite usar rand() ni <random> para la lógica del juego):

```
x(k+1) = (1103515245 · x(k) + 12345) mod 2^31
semilla de la oleada n = semilla_partida · 1000 + n
```

- La oleada n contiene 6 + 3·n apariciones y se genera completa al iniciar la ronda, como una ColaSpawn de nodos {tick_aparicion, tipo, color, ángulo, sentido}.
- Conjunto de tipos disponible: *[Dron, Dron]*; se agrega Girador si n ≥ 2; Divisor si n ≥ 3; Tanque si n ≥ 4; Camaleón y Girador si n ≥ 5. En ese orden.
- Por cada aparición se consumen números en este orden: tipo = x mod tamaño_del_conjunto; color = x mod 3; ángulo = x mod 360; sentido (solo Girador) = x mod 2 (0 = horario).
- La primera aparición ocurre en el tick 60. Intervalo base = máx(9, 35 − 2·n) ticks; intervalo real = ⌊base · (60 + x mod 81) / 100⌋.
- Desde la oleada 7, si x mod 4 = 0 la siguiente aparición ocurre en el mismo tick (aparición doble).
- La oleada termina cuando la cola está vacía y la lista de enemigos también.

3.7 Orden de procesamiento de cada tick

El orden es obligatorio; alterarlo cambia los resultados y hace fallar las pruebas de reproducibilidad.

1. Leer las acciones del tick (teclado, script o bot): girar hacia un ángulo objetivo, cambiar color, activar pulso.
2. Actualizar el ángulo de la nave (máximo 6° por el camino más corto).
3. Desencolar de ColaSpawn todo enemigo con tick_aparicion ≤ tick actual e insertarlo al final de ListaEnemigos.
4. Disparo automático si tick mod 4 = 0: insertar la bala al inicio de ListaBalas.
5. Mover enemigos (r, θ) y actualizar el color de los Camaleones.
6. Resolver choques enemigo–nave y enemigo–núcleo.
7. Mover balas, eliminar las que superen r = 320 y resolver impactos bala–enemigo.
8. Avanzar la onda de pulso y resolver sus bajas.
9. Registrar eventos y actualizar estadísticas.
10. Verificar fin de oleada y fin de partida del jugador.

4. Estructuras de datos obligatorias

Todas las estructuras de esta sección deben implementarse con nodos y punteros propios. Cada clase gestiona su memoria (destructor que libera todos los nodos) y declara explícitamente su política de copia.

Estructura	Tipo	Uso en el juego	Operaciones exigidas	Complejidad
ListaEnemigos	Doblemente enlazada con cabeza y cola	Enemigos activos en pantalla	insertarFinal, insertarDespues, eliminar(nodo) que retorna el siguiente, recorrido en ambos sentidos	O(1) cada operación; recorrido O(n)

Estructura	Tipo	Uso en el juego	Operaciones exigidas	Complejidad
ListaBalas	Simplemente enlazada	Balas activas	insertarInicio; eliminarSi(predicado) usando NodoBala**	O(1) inserción; O(n) depuración en una pasada
ColaSpawn	Cola enlazada (frente y final)	Apariciones pendientes de la oleada	encolar, desencolar, verFrente, constructor de copia y operador= con copia profunda	O(1); copia O(n)
RondaJugadores	Circular doblemente enlazada	Orden de turnos	agregar, avanzar(horario / antihorario), eliminarActual, recorrido completo	O(1); recorrido O(n)
BitacoraEventos	Simplemente enlazada con puntero a cola	Eventos de la partida para estadísticas	agregarFinal, recorrido, filtrado por jugador y oleada	O(1) inserción
Ranking	Lista simple ordenada	Tabla de mejores jugadores	insertarOrdenado, top(k)	O(n) inserción
HistorialPartidas	Simplemente enlazada	Registros cargados desde CSV	agregarFinal, copiar valores a lista auxiliar, merge sort	O(n log n) ordenamiento

4.1 Requisitos específicos

- **Eliminación durante recorrido:** al eliminar enemigos o balas dentro del ciclo que los recorre no se puede saltar ni revisitar ningún nodo. La depuración de balas debe usar puntero a puntero (NodoBala**) sin caso especial para la cabeza.
- **Divisor:** los dos Mini se insertan con insertarDespues sobre el nodo del Divisor antes de eliminarlo, de modo que queden en la posición que ocupaba.
- **Copia profunda:** la cola de cada oleada se genera una sola vez por ronda y cada jugador recibe una copia independiente. Consumir la copia no puede alterar la original.
- **Regla de tres:** toda clase con memoria dinámica implementa destructor, constructor de copia y operador de asignación, o los elimina explícitamente con = delete y justifica la decisión.
- **Sin fugas:** al terminar el programa no puede quedar memoria reservada sin liberar.

5. Modo multijugador

5.1 Modo por turnos (obligatorio)

Participan de 1 a 6 jugadores, cada uno humano o bot. Cada jugador tiene su propio núcleo, puntaje, combo y carga de pulso. Los jugadores se almacenan en RondaJugadores en el orden en que se registran.

1. Al iniciar la ronda k se genera la ColaSpawn de la oleada k una sola vez.
2. Cada jugador activo juega la oleada k completa con una copia profunda de esa cola. El combo se reinicia al inicio de cada turno; la vida del núcleo y la carga de pulso se conservan.
3. Orden serpenteante: en rondas impares la ronda avanza en sentido horario (siguiente) y en rondas pares en sentido antihorario (anterior). Cada ronda comienza con el último jugador que jugó la ronda anterior.
4. Si el núcleo de un jugador llega a 0, este se elimina de la ronda con eliminarActual y sus estadísticas se cierran.
5. La partida termina cuando queda un solo jugador activo tras completar una ronda, cuando se alcanza el límite de oleadas configurado o cuando no quedan jugadores. Gana el jugador con mayor puntaje; en empate, la mayor oleada alcanzada y luego la mayor precisión.

5.2 Jugador automático (bot, obligatorio)

El bot permite probar el juego sin intervención humana y generar suficientes partidas para el módulo de estadísticas. Su heurística es fija para que los resultados sean comparables entre grupos:

- Objetivo: el enemigo con menor r entre los que cumplen r > 100. En empate, el primero de la lista.
- Acción: girar hacia el θ del objetivo y cambiar a su color si es distinto al actual.
- Pulso: se activa si la carga es 100 y hay al menos 5 enemigos con r < 160.
- Sin objetivo: mantiene ángulo y color.

5.3 Modo cooperativo simultáneo (opcional, bonificación)

Dos naves comparten el anillo y el núcleo durante la misma oleada, cada una con su propia fase de color. Cada bala guarda el identificador de su dueño y las bajas, impactos erróneos y puntos se atribuyen al jugador que disparó. Los daños al núcleo se registran como compartidos. Este modo es opcional y vale hasta 5 puntos adicionales sobre la nota de 100.

6. Módulo de estadísticas

6.1 Métricas por jugador y partida

Métrica	Definición
Puntaje	Suma de puntos obtenidos
Oleada alcanzada	Última oleada iniciada por el jugador
Disparos	Balas emitidas
Impactos correctos	Impactos con color igual al del enemigo
Impactos erróneos	Impactos con color distinto
Precisión de color	correctos / (correctos + erróneos)

Métrica	Definición
Eficiencia de disparo	correctos / disparos
Precisión por color	Para C, M y A: correctos con esa fase / impactos totales con esa fase
Bajas por tipo y por color	Conteos separados; incluye bajas por pulso
Combo máximo	Mayor valor alcanzado por el combo
Daño recibido	Total, y desglosado por tipo de enemigo que lo causó
Pulsos usados	Cantidad de activaciones
Ticks sobrevividos	Suma de ticks jugados en todos sus turnos
Posición final	Lugar obtenido en la partida

6.2 Bitácora de eventos

Cada evento relevante se agrega a BitacoraEventos con la forma {tick, oleada, jugador, tipo_evento, tipo_enemigo, color, valor}. Los tipos de evento son BAJA, IMPACTO_ERRONEO, DANO_NUCLEO, CHOQUE_NAVE, PULSO, FIN_OLEADA y ELIMINACION. Los disparos se contabilizan, pero no se registran como eventos individuales. Al final de cada partida el programa imprime, por jugador, una línea de tiempo por oleada con bajas, daño recibido y puntaje acumulado, calculada recorriendo la bitácora.

6.3 Estadísticas históricas

Las partidas se acumulan en un archivo CSV. A partir de él se calcula, por jugador:

- Partidas jugadas, victorias, mejor y peor puntaje.
- Media, mediana y desviación estándar poblacional del puntaje: $\sigma = \sqrt{(\Sigma(x_i - \mu)^2 / N)}$.
- La mediana se obtiene copiando los puntajes a una lista enlazada auxiliar y ordenándola con **merge sort sobre la lista** (sin pasar a arreglo). Con N par, la mediana es el promedio de los dos centrales.
- Precisión de color promedio y color con mejor precisión.
- Tipo de enemigo que más daño le ha causado.
- Histograma ASCII de puntajes en 10 intervalos de igual ancho entre el mínimo y el máximo.

6.4 Ranking

Lista ordenada construida con inserción ordenada. Criterios: mejor puntaje descendente; en empate, mayor precisión de color; luego orden alfabético del alias. Se muestran los 10 primeros.

6.5 Persistencia

Archivo historial.csv, separado por comas, con encabezado y una fila por jugador y partida:

```
id_partida,semilla,modo,jugador,es_bot,posicion,puntaje,oleada,disparos,correctos,erroneos,
combo_max,pulsos,dano_total,ticks,bajas_dron,bajas_girador,bajas_divisor,bajas_mini,
bajas_tanque,bajas_camaleon
```

Al iniciar, el programa carga el historial en HistorialPartidas; al terminar cada partida agrega sus filas. Filas mal formadas se reportan con su número de línea y se omiten sin detener el programa.

7. Interfaz y ejecución

7.1 Modos de ejecución

Modo	Descripción	Requisito
Interfaz gráfica Allegro 5	Modo principal de juego para humanos y bots, en tiempo real, con menú, registro de jugadores, partida, cambio de turno, fin de partida, estadísticas y ranking.	Obligatorio
Consola por lotes	Ejecuta la partida completa a partir de un script de acciones o con bots, sin ventana ni intervención humana. Imprime el reporte al final. Se usa para las pruebas de reproducibilidad.	Obligatorio
Reportes por consola	Impresión del reporte de estadísticas y del ranking a partir de historial.csv.	Obligatorio

7.2 Línea de comandos

```
./faseorbital                                  # abre la interfaz Allegro 5
./faseorbital --lotes --jugadores Ana,Luis,BOT1,BOT2 --semilla 42 --oleadas 10
./faseorbital --lotes --jugadores Ana,Luis --semilla 7 --script partida.txt
./faseorbital --estadisticas historial.csv
./faseorbital --ranking historial.csv
```

Todo alias que comience con BOT se controla con la heurística de la sección 5.2, tanto en el modo gráfico como en el de lotes.

7.3 Formato del script de acciones

Una acción por línea: jugador, oleada, tick y acción. Los ticks no listados mantienen el estado anterior. Las líneas que empiezan con # son comentarios.

```
# jugador oleada tick ACCION [parametro]
Ana 1 0   COLOR M
Ana 1 75  GIRAR_A 135
Ana 1 140 COLOR A
Ana 2 300 PULSO
Luis 1 10 GIRAR_A 45
```

7.4 Requisitos de la interfaz Allegro 5

Aspecto	Requisito
Bucle principal	Cola de eventos (ALLEGRO_EVENT_QUEUE) con un temporizador (ALLEGRO_TIMER) a 30 eventos por segundo. Cada evento del temporizador ejecuta exactamente un tick del Motor; el dibujo ocurre solo cuando la cola de eventos está vacía.

Aspecto	Requisito
Ventana	Tamaño mínimo 800 × 600, con cierre correcto desde el botón de la ventana (ALLEGRO_EVENT_DISPLAY_CLOSE).
Dibujo	Addon de primitivas: núcleo con arco de vida, anillo de órbita, nave orientada con el color de su fase, balas, los seis tipos de enemigo con forma distinta y color de fase, onda de pulso y partículas o destello en cada baja.
Controles	Mouse o flechas izquierda y derecha para girar; teclas 1, 2 y 3 para las fases C, M y A; espacio para el pulso; P o Escape para pausar.
Información en pantalla	Addons de fuente y TTF: jugador en turno, puntaje, multiplicador, oleada, vida del núcleo y carga de pulso.
Pantallas	Menú principal, registro de jugadores (humano o bot), anuncio de turno, partida, pausa, resumen de oleada, fin de partida con posiciones, estadísticas históricas y ranking top 10.
Audio	Addons de audio y códecs: al menos un efecto para disparo o baja, impacto erróneo, daño al núcleo y pulso.
Recursos	Toda fuente, sonido, temporizador, cola y ventana creados se destruyen al salir. Si falta un recurso, el programa informa cuál y termina sin caerse.
Separación	La vista solo lee el estado del Motor y le envía acciones. Ninguna regla del juego puede vivir en el código de Allegro.

8. Arquitectura sugerida

Módulo	Responsabilidad
Generador	Generador congruencial lineal de la sección 3.6
ListaEnemigos, ListaBalas, ColaSpawn	Estructuras del motor
RondaJugadores, Jugador	Estado de cada jugador y rotación de turnos
Oleada	Genera la ColaSpawn de la oleada n a partir de la semilla
Motor	Ejecuta un tick y un turno completo según la sección 3.7
Controlador	Fuente de acciones: teclado y mouse de Allegro, script o bot
VistaAllegro	Inicialización de Allegro 5, bucle de eventos, dibujo, pantallas, audio y liberación de recursos
BitacoraEventos, Estadisticas	Registro de eventos y cálculo de métricas
HistorialPartidas, Ranking, Persistencia	Carga y guardado del CSV, ordenamiento y ranking
main	Lectura de argumentos y selección de modo

El mismo Motor debe funcionar en el modo gráfico y en el modo por lotes. Por eso el modo por lotes no puede depender de Allegro.

9. Restricciones técnicas

- VISUAL C++ 2026 COMMUNITYcon Allegro 5.2.x.
- Addons de Allegro permitidos: primitives, font, ttf, image, audio y acodec.
- **Prohibido** usar std::vector, std::list, std::forward_list, std::deque, std::queue, std::stack, std::map, std::set, std::unordered_*, std::priority_queue, std::sort y punteros inteligentes en las estructuras y algoritmos exigidos.
- Permitido: std::string, iostream, fstream, sstream, cmath, cstdint, iomanip y funciones auxiliares de cadenas.
- Memoria dinámica con new y delete. Se verificará con Valgrind (memcheck) o AddressSanitizer sobre el modo por lotes.
- La aleatoriedad del juego proviene exclusivamente del generador de la sección 3.6. Los efectos visuales (partículas) pueden usar otra fuente.
- El uso de una estructura prohibida en un componente obligatorio anula el puntaje de ese criterio.

10. Casos de prueba mínimos

El grupo debe entregar un programa de pruebas que ejecute al menos los siguientes casos y reporte cuáles pasan.

ID	Componente	Caso	Resultado esperado
P01	ListaEnemigos	Insertar 1000 nodos, eliminar los de id par durante un recorrido	Tamaño 500; recorrido hacia adelante y hacia atrás coinciden
P02	ListaEnemigos	Eliminar cabeza, cola y nodo único	Punteros cabeza y cola correctos; lista vacía con ambos en nulo
P03	ListaEnemigos	insertarDespues sobre el último nodo	La cola se actualiza al nodo nuevo
P04	ListaBalas	Eliminar balas consecutivas, incluida la cabeza, con NodoBala**	Ninguna bala marcada sobrevive; ninguna válida se pierde
P05	ColaSpawn	Encolar y desencolar hasta vaciar	Frente y final nulos; desencolar en vacía retorna falso
P06	ColaSpawn	Copiar, consumir la copia, destruir ambas	Original intacta; sin doble liberación
P07	RondaJugadores	Eliminar el jugador actual, el único jugador y rotar en ambos sentidos	Enlaces circulares válidos en todo momento
P08	Motor	Dos ejecuciones con la misma semilla y script	Reportes idénticos (comparación con diff)

ID	Componente	Caso	Resultado esperado
P09	Motor	Impacto con color incorrecto	r aumenta 9; combo = 0; vida sin cambio
P10	Motor	Baja de un Divisor	Dos Mini insertados en su posición, con los dos colores distintos al del padre
P11	Motor	Pulso con carga menor a 100 y con carga 100	Sin efecto en el primer caso; bajas en la banda en el segundo
P12	Estadísticas	Merge sort con lista vacía, 1, 2, ordenada, inversa y con repetidos	Orden correcto; mediana correcta con N par e impar
P13	Persistencia	Guardar, reiniciar el programa y cargar	Mismos resultados en estadísticas y ranking; líneas inválidas reportadas
P14	Memoria	Partida con 4 bots y 10 oleadas bajo Valgrind	0 bytes perdidos y 0 errores

11. Entregables

Entregable	Contenido
Código fuente	Repositorio o archivo comprimido con Makefile o CMake, recursos (fuentes y sonidos) y README con instrucciones de compilación
Programa de pruebas	Ejecutable que corre los casos P01 a P14 y muestra el resultado de cada uno
Documentación técnica (PDF, máximo 12 páginas)	Diagramas de cada estructura; tabla de complejidad; decisiones de diseño y política de copia; arquitectura y separación Motor–Vista; evidencia de Valgrind o AddressSanitizer y de las pruebas
Manual de usuario (PDF)	Requisitos e instalación de Allegro 5; compilación y ejecución; reglas y controles; modos de juego; lectura de estadísticas y ranking; capturas de pantalla
Video de defensa (10 a 15 minutos)	Demostración del juego en Allegro 5 con al menos 3 jugadores (incluido un bot), estadísticas y ranking; explicación sobre el código de las estructuras enlazadas; participación de todos los integrantes con cámara o voz identificable
Evidencia de ejecución	historial.csv con al menos 30 partidas generadas con bots y el reporte de estadísticas producido a partir de él

12. Rúbrica de revisión

La nota se compone de **Documentación (25 %)** y **Ejecución (75 %)**. Cada criterio se califica con los niveles de la sección 12.4 y el puntaje obtenido es el peso multiplicado por el porcentaje del nivel.

12.1 Resumen de pesos

Componente	Subcomponente	Peso
Documentación	Documentación técnica	12,5 %
	Manual de usuario	5,5 %
	Video de defensa	7 %
	Subtotal documentación	**25 %**
Ejecución	Funciones del videojuego	40 %
	Estadísticas	17 %
	Interfaz con Allegro 5	18 %
	Subtotal ejecución	**75 %**
Total		**100 %**

12.2 Documentación (25 %)

ID	Criterio	Peso	Obtenido	Observaciones
DT1	Técnica: diagramas de nodos y enlaces de cada estructura	4		
DT2	Técnica: tabla de complejidad de todas las operaciones, justificada	3,5		
DT3	Técnica: arquitectura, separación Motor–Vista, decisiones de diseño y política de copia	2,5		
DT4	Técnica: evidencia de pruebas P01 a P14 y de memoria sin fugas	2,5		
	Subtotal documentación técnica	**12,5**		
DU1	Usuario: requisitos, instalación de Allegro 5 y compilación	1,5		
DU2	Usuario: reglas del juego, controles y modos	2		
DU3	Usuario: estadísticas y ranking explicados con capturas	2		
	Subtotal manual de usuario	**5,5**		
DV1	Video: demostración completa del juego, multijugador y estadísticas	2,5		

ID	Criterio	Peso	Obtenido	Observaciones
DV2	Video: explicación sobre el código de las estructuras y sus punteros	3		
DV3	Video: participación de todos los integrantes, claridad y duración de 10 a 15 minutos	1,5		
	Subtotal video de defensa	**7**		
	Total documentación	**25**		

12.3 Ejecución (75 %)

Funciones del videojuego (40 %)

ID	Criterio	Peso	Obtenido	Observaciones
EJ1	ListaEnemigos, ListaBalas, ColaSpawn y BitacoraEventos correctas en casos borde, con eliminación durante recorrido	10		
EJ2	Reglas del juego: órbita y giro, disparo automático, fases de color, retroceso, combo, pulso, daño, oleadas y orden del tick	10		
EJ3	Los seis tipos de enemigo con su comportamiento, incluida la división y el cambio de color	5		
EJ4	Multijugador por turnos: ronda circular doble, orden serpenteante, copia profunda de la oleada, eliminación de jugadores y ganador	8		
EJ5	Bot con la heurística especificada, en modo gráfico y por lotes	3		
EJ6	Gestión de memoria: regla de tres, cero fugas y cero errores; reproducibilidad con semilla (P08)	4		
	Subtotal funciones del videojuego	**40**		

Estadísticas (17 %)

ID	Criterio	Peso	Obtenido	Observaciones
ES1	Métricas por jugador y partida, bitácora de eventos y línea de tiempo por oleada	5		
ES2	Merge sort sobre lista enlazada, media, mediana, desviación estándar e histograma	5		
ES3	Ranking con inserción ordenada y criterios de desempate	3		
ES4	Persistencia en historial.csv: carga, guardado y manejo de líneas inválidas	4		

ID	Criterio	Peso	Obtenido	Observaciones
	Subtotal estadísticas	**17**		

Interfaz con Allegro 5 (18 %)

ID	Criterio	Peso	Obtenido	Observaciones
AL1	Inicialización, cola de eventos, temporizador a 30 ticks por segundo y liberación de todos los recursos	4		
AL2	Dibujo con primitivas: núcleo, anillo, nave, balas, seis enemigos distinguibles, onda de pulso y efectos de baja	5		
AL3	Controles con teclado y mouse, pausa y cierre de ventana	3		
AL4	Información en pantalla y pantallas de menú, registro, turno, fin de partida, estadísticas y ranking	4		
AL5	Efectos de sonido con los addons de audio	2		
	Subtotal Allegro 5	**18**		
	Total ejecución	**75**		

12.4 Niveles de desempeño

Nivel	Porcentaje del peso	Descripción
Completo	100 %	Implementado según la especificación y funciona correctamente en todos los casos revisados
Satisfactorio	75 %	Implementado completo con defectos menores que no afectan el resultado del juego ni de las estadísticas
Parcial	50 %	Implementado en parte, o con errores que afectan algunos casos
Insuficiente	25 %	Existe un intento, pero no funciona en la mayoría de los casos
Ausente	0 %	No se entregó, no compila o usa estructuras prohibidas

12.5 Bonificación y penalizaciones

- **Bonificación:** modo cooperativo simultáneo (sección 5.3), hasta 5 puntos adicionales.
- El proyecto no compila con las instrucciones del README: la ejecución vale 0.
- Uso de estructuras prohibidas en un componente obligatorio: 0 en el criterio correspondiente.
- Sin video de defensa, las funciones que no se demuestren en él pueden calificarse como no verificadas.
- Entrega tardía: No permitida.

- Un integrante que no participe en el video o no demuestre dominio del código puede recibir una nota individual inferior o Nula, a la del grupo.

13. Cronograma sugerido

Semana	Hito	Resultado verificable
1	Estructuras base	Listas, cola y ronda circular con pruebas P01 a P07 pasando
2	Motor y bot	Partida individual completa en modo por lotes; P08 a P11
3	Interfaz Allegro 5 y multijugador	Juego jugable en ventana con turnos, bots y todas las pantallas de partida
4	Estadísticas, persistencia y documentación	Reportes, CSV, ranking en pantalla, P12 a P14, documentación técnica, manual y video

Fechas exactas: *[por definir]*.

14. Normas académicas

- El código debe ser original del grupo. Se aplicarán herramientas de detección de similitud entre entregas.
- Uso de asistentes de programación o inteligencia artificial: prohibido.
- Toda fuente externa consultada debe citarse en el documento técnico. Por ejemplo los materiales y casos del Laboratorio Asincrónico de Allegro y VideoJuegos…o similares.

Anexo A. Interfaces de referencia

Las siguientes declaraciones son una guía. Se pueden agregar métodos, pero los listados deben existir con semántica equivalente.

```
enum class TipoEnemigo { Dron, Girador, Divisor, Mini, Tanque, Camaleon };

struct Enemigo {
    int id; TipoEnemigo tipo; int color;      // 0 = C, 1 = M, 2 = A
    double r, theta, omega; int vida, radio; int ticksColor;
};
struct NodoEnemigo { Enemigo dato; NodoEnemigo* ant; NodoEnemigo* sig; };

class ListaEnemigos {
    NodoEnemigo* cabeza; NodoEnemigo* cola; int tam;
public:
    ListaEnemigos(); ~ListaEnemigos();
    ListaEnemigos(const ListaEnemigos&) = delete;
    ListaEnemigos& operator=(const ListaEnemigos&) = delete;
    NodoEnemigo* insertarFinal(const Enemigo& e);                    // O(1)
    NodoEnemigo* insertarDespues(NodoEnemigo* n, const Enemigo& e);  // O(1)
    NodoEnemigo* eliminar(NodoEnemigo* n);          // O(1), retorna n->sig
    NodoEnemigo* primero() const; NodoEnemigo* ultimo() const;
    int tamano() const; bool vacia() const;
};
```

```
struct Bala { double x, y, vx, vy; int color; int duenio; };
struct NodoBala { Bala dato; NodoBala* sig; };
class ListaBalas {
    NodoBala* cabeza; int tam;
public:
    void insertarInicio(const Bala& b);                              // O(1)
    template <class Pred> int eliminarSi(Pred debeEliminarse);       // usa NodoBala**
};

struct Spawn { int tick; TipoEnemigo tipo; int color; int angulo; int sentido; };
class ColaSpawn {
    struct Nodo { Spawn dato; Nodo* sig; }; Nodo* frente; Nodo* final; int tam;
public:
    ColaSpawn(); ~ColaSpawn();
    ColaSpawn(const ColaSpawn& otra);                                // copia profunda
    ColaSpawn& operator=(const ColaSpawn& otra);
    void encolar(const Spawn& s); bool desencolar(Spawn& out);
    const Spawn* verFrente() const; bool vacia() const;
};

class RondaJugadores {
    struct Nodo { Jugador* j; Nodo* ant; Nodo* sig; }; Nodo* actual; int tam;
public:
    void agregar(Jugador* j);                   // antes de actual (al "final")
    void avanzar(bool horario);
    Jugador* eliminarActual(bool horario);      // retorna el eliminado
    Jugador* verActual() const; int tamano() const;
};

class Generador {
    uint32_t x;
public:
    explicit Generador(uint32_t semilla) : x(semilla) {}
    uint32_t siguiente() { x = (1103515245u * x + 12345u) & 0x7FFFFFFFu; return x; }
};
```

Anexo B. Glosario

Término	Significado
Tick	Unidad discreta de tiempo de la simulación
Fase	Color activo de la nave
Combo	Bajas consecutivas con color correcto sin recibir daño ni fallar color
Oleada	Conjunto de enemigos generados a partir de una misma semilla
Ronda	Una oleada jugada por todos los jugadores activos
Copia profunda	Duplicado que reserva nodos nuevos en lugar de compartir punteros
Pulso	Habilidad que destruye enemigos sin importar su color

Anexo C. Videos Demostrativos/Explicativos

1. FaseOrbital_1_primeras_oleadas **(33 s):** oleadas 1 y 2 con drones y giradores. Se ve cómo la jugadora cambia de color según el enemigo y cómo el combo sube
2. FaseOrbital_2_oleadas_avanzadas_y_pulso **(48 s):** oleadas 5 y 6 con tanques, divisores que se parten en minis y camaleones que cambian de color. Cerca del segundo 19 se activa el pulso y barre la pantalla.
3. FaseOrbital_3_multijugador_por_turnos **(70 s):**
   - Ronda 3 con Ana, BOT1 y Luis. Las rondas 1 y 2 se simularon antes, así que los puntajes que llegan a esta ronda son reales.
   - Cada jugador enfrenta la misma oleada, con una tarjeta de turno y un marcador lateral de puntaje y núcleo.
   - Termina con la tabla de resultados (precisión, combo máximo y bajas). Ganó BOT1.
4. FaseOrbital_4_fin_de_partida **(27 s):** una jugadora más débil llega a la oleada 6, el núcleo cae y aparece la pantalla de fin con su puntaje y estadísticas.
5. FaseOrbital_5_tutorial_controles_y_reglas **(55 s).** Seis pasos numerados, cada uno con recuadros que señalan lo que ocurre:
   - La nave, el anillo, el núcleo y el disparo automático.
   - Color correcto: se ve cuál es la fase activa y la baja.
   - Color incorrecto: el enemigo retrocede 9 u, el combo se rompe y luego se cambia a Magenta.
   - Combo y multiplicador, con los valores actualizándose en vivo.
   - Pulso: carga completa, activación y la onda en cámara lenta.
   - Daño al núcleo y choque con la nave.

   Termina con un resumen de reglas y controles.
6. FaseOrbital_6_catalogo_de_enemigos **(65 s, a ×0,5).** Presenta cada tipo con una ficha lateral (vida, radio, velocidad, puntos, oleada en que aparece):
   - **Girador:** deja una estela para que se vea la espiral.
   - **Divisor:** se pausa justo cuando ocurre el insertarDespues que crea los dos Mini.
   - **Tanque:** muestra cuánta vida le queda.
   - **Camaleón:** muestra una cuenta regresiva hasta su próximo cambio de color.

   Cierra con la tabla resumen de los seis tipos
7. FaseOrbital_7_estructuras_de_datos_en_vivo **(60 s, a ×0,5).** Es el más útil para el curso: el juego a la izquierda y la memoria a la derecha, en tiempo real.
   - **ListaEnemigos:** nodos con enlaces sig/ant, cabeza y cola, y los nodos nuevos resaltados. Cada enemigo lleva en pantalla su número (#id) para relacionarlo con su nodo.
   - **ColaSpawn:** frente y final de la cola.
   - **ListaBalas:** crece por la cabeza con insertarInicio.
   - **Operaciones y bitácora:** contadores acumulados y los eventos de BitacoraEventos.

   Termina con la tabla de complejidad de cada operación.
8. FaseOrbital_8_multijugador_y_estadisticas **(82 s).**
   - **Partida:** se registran 4 jugadores y se va armando la lista circular doble. La partida corre a ×10, con el diagrama mostrando el jugador actual y el cambio de sentido en cada ronda. Luis queda eliminado y se ve cómo sale de la lista con eliminarActual().
   - **Resultados:** tabla de métricas por jugador y línea de tiempo por oleada de la ganadora.
   - **Historial:** 40 partidas en historial.csv; estadísticas de Ana (media, mediana, desviación estándar) con histograma; ranking final.